```python
import math
import random

from learning_module import LearningModule, Lesson, Quiz
from users.user import User


class Educator(User):
    """
    This is the class definition for the Educator class.
    """

    def __init__(self, email, first_name, last_name, username, password,
    date_of_birth):
        """
        Constructor for the Educator class.

        :param email: The email address of the educator.
        :param first_name: The first name of the educator.
        :param last_name: The last name of the educator.
        :param username: The username of the educator.
        :param password: The password of the educator.
        :param date_of_birth: The date of birth of the educator.
        """
        super().__init__(
            email, first_name, last_name, username, password, role="Educator"
        )
        # Educator ID is a unique identifier for each educator
        self.educatorID = (
            first_name + last_name + str(math.floor(1000 * random.random()))
        )
        self.date_of_birth = date_of_birth
        self.feedbacks = []  # Feedbacks this specific educator has received
        self.modules = []  # Learning modules created by this educator

    def dashboard(self, login_window=None) -> None:
        """
        This method is used to display the dashboard for the educator.

        :param login_window: The login window to be destroyed after successful login.
        """
        if login_window:
            login_window.destroy()

        while True:
            print(f"\nWelcome {self.first_name}\n")
            print("1. Create Learning Module")
            print("2. Edit Learning Module")
            print("3. Feedbacks")
            print("4. Edit Profile")
            print("5. Logout")
            choice = input("Enter your choice: ")

            if choice == "1":
                self.create_learning_module()
            elif choice == "2":
                chosen_module = None
                # Displaying the list of modules created by this educator
                print("\nChoose a module to edit\n")
                for i, module in enumerate(self.modules):
                    print(f"{i+1}. {module.get_title()}")
                choice = int(input("Enter your choice: "))

                chosen_module = self.modules[choice - 1]
                self.edit_learning_module(chosen_module)
            elif choice == "3":
                self.view_feedbacks()
            elif choice == "4":
                self.edit_profile()
            elif choice == "5":
                break
            else:
                print("Please choose a valid option.")
```

```
73
74      def create_lessons(self) -> list[Lesson]:
75          """
76          Creates new lessons and returns it.
77
78          :return: A list of lessons created.
79          """
80          lessons = []
81          while True:
82              print("\nCreate lesson\n")
83              title = input("Enter the title of the lesson: ")
84              content = input("Enter the content of the lesson: ")
85              lesson = Lesson(title, content)
86              lessons.append(lesson)
87              choice = input("Do you want to add another lesson? (Y/N): ")
88              if choice.lower() == "n":
89                  break
90
91          return lessons
92
93      def create_quizzes(self) -> list[Quiz]:
94          """
95          Creates new quizzes and returns it.
96
97          :return: A list of quizzes created.
98          """
99          quizzes = []
100         while True:
101             print("\nCreate quiz\n")
102             question = input("Enter the question: ")
103             options = []
104             for i in range(4):
105                 options.append(input(f"Enter option {i+1}: "))
106             answer = int(input("Enter the answer (1-4): ")) - 1
107             hint = input("Enter the hint: ")
108             solution = input("Enter the solution: ")
109             quiz = Quiz(question, options, answer, hint, solution)
110             quizzes.append(quiz)
111             choice = input("Do you want to add another quiz? (Y/N): ")
112             if choice.lower() == "n":
113                 break
114
115         return quizzes
116
117     def create_learning_module(self) -> None:
118         """
119         Creates a new learning module and adds it to the text file of learning
            modules.
120         """
121         print("\nCreate learning module\n")
122         title = input("Enter the title of the module: ")
123         description = input("Enter the description of the module: ")
124         lessons = self.create_lessons()
125         quizzes = self.create_quizzes()
126         difficulty = int(input("Enter the difficulty of the module (1-5): "))
127         achievement = input(
128             "Enter the achievement for this module (press enter to skip): "
129         )
130         print("\nLearning module created successfully!")
131
132         new_module = LearningModule(
133             title, description, lessons, quizzes, difficulty, achievement
134         )
135         self.modules.append(new_module)
136
137         # Need to add the new module to database
138
139     def edit_learning_module(self, module: LearningModule) -> None:
140         """
141         Edit a learning module.
142
143         :param module: The module to be edited.
144         """
```

```
145             print(f"\nEditing {module.get_title()}\n")
146             print("1. Edit lesson")
147             print("2. Edit quiz")
148             print("3. Go back")
149             choice = input("Enter your choice: ")
150 
151             if choice == "1":
152                 module.edit_lesson()
153             elif choice == "2":
154                 module.edit_quiz()
155             elif choice == "3":
156                 return
157             else:
158                 print("Please choose a valid option.")
159 
160         def view_feedbacks(self) -> None:
161             """
162             View feedbacks received by this educator.
163             """
164             print("\nFeedbacks\n")
165             for feedback in self.feedbacks:
166                 print(feedback)
167 
168         def __str__(self):
169             return f"{self.first_name} {self.last_name} ({self.username})"
170 
```